DevOpsCon

by entwickler.de

Reproducible Builds

Robots recreate Electric Sheep

Brett Smith
xbcsmith@gmail.com
20240405

Introduction to Secure Supply Chain

Define a Secure Supply Chain:

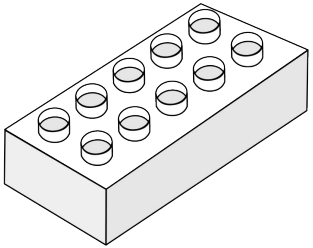
A secure supply chain, in the context of software, encompasses the end-to-end process through which software is developed, tested, and delivered to end-users or customers. It includes all the steps from initial code development to distribution and maintenance.

Importance of Supply Chain Security:

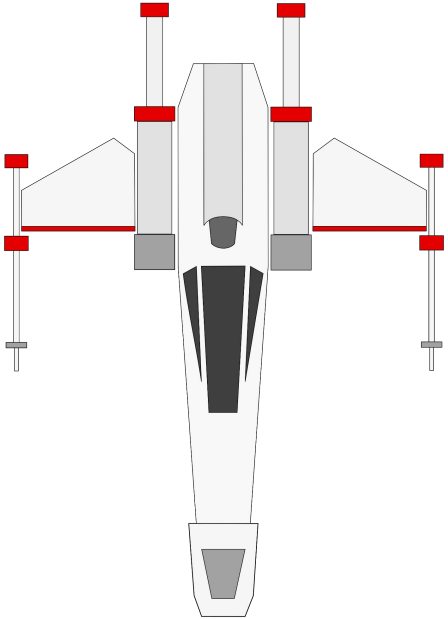
The security of the software supply chain is vital because it directly impacts the safety and reliability of the software products that we use daily. A breach or vulnerability in the supply chain can have far-reaching consequences, from data breaches to system failures.

The Role of Software in Supply Chains:

It's crucial to understand that software is not only a component within the supply chain; it often controls and orchestrates the entire process. From managing inventory in manufacturing to coordinating logistics in transportation, software is an integral part of modern supply chains.



What are Reproducible Builds?



Lego X-Wing fighter

Process:

1. **Setup the place to build the kit**
2. **Use the parts from the kit**
3. **Follow the assembly instructions from the kit**

Results:

You get the exact same X-Wing fighter Lego intended you to get.

When is a build reproducible?

The Debian definition of “reproducible builds” in reproducible-builds.org initiative 2:

A build is reproducible if given the same source code, build environment, and build instructions, any party can recreate bit-by-bit identical copies of all specified artifacts.

The relevant attributes of the build environment, the build instructions, and the source code as well as the expected reproducible artifacts are defined by the authors or distributors. The artifacts of a build are the parts of the build results that are the desired primary output.

How do Reproducible Builds Ensure Integrity?

Determinism

Build Process that is Predictable and Consistent

Difficult to introduce changes or vulnerabilities to the process

Unauthorized modifications are immediately apparent

Levels of Deterministic Builds

Deterministic builds produces exactly the same output every time.

Basic determinism:

- full build
- same source code
- same directory
- same machine

Incremental determinism:

- partial rebuild
- same source code
- same directory
- same machine

Local determinism:

- full build
- same source code
- any directory
- same machine

Universal determinism:

- full build
- same source code
- any directory
- any machine the build runs on.

Things that wreck reproducible builds

Build Environment

Timestamps

Files

Networks

Locale

Dependencies

Randomness

Packaging

Pillars of Reproducible Builds

- Repeatable builds
- Immutable environments
- Source availability

Deterministic Build System

Define the Build Environment

Distribute the Build Environment

Provide the Comparison Protocol

Things we can do to make builds reproducible

Infrastructure as Code and gitOps

Containerize the toolchains, immutable is good.

Pin your versions

Record the build environment

Checksums and Cryptographic Signatures

Hermetic builds

Remove randomness from the resulting binary.

Build Systems

That try to meet the definition of reproducible build:

- Debian buildd system

That can never meet the definition of reproducible build:

- Fedora Buildsystem

The Debate

Is the Debian definition of reproducible builds the only way?

The Ideal Reproducible Build

More than one pipeline build the same source

Wrap up

Are the toolchains containerized?

Are the build environments immutable?

Are the build inputs always available?

Are the sources verifiable?

Are the builds instructions repeatable?

AMA

I am Smitty and I am afraid of Robots

Brett Smith
xbcsmith@gmail.com
20240405

Resources

Web Articles

- https://reproducible-builds.org
- https://lists.debian.org/debian-release/2023/09/msg00625.html
- https://lists.reproducible-builds.org/pipermail/rb-general/2023-September/003076.html
- https://buildinfos.debian.net/ftp-master.debian.org/buildinfo/2023/09/16/
- https://fossa.com/blog/three-pillars-reproducible-builds/
- https://blog.llvm.org/2019/11/deterministic-builds-with-clang-and-lld.html
- https://discussion.fedoraproject.org/t/report-from-the-reproducible-builds-hackfest-during-flock-2023/87469

Interesting tools

- https://diffoscope.org/
- https://salsa.debian.org/reproducible-builds/debian-verification-build-experiment

Please,

don't forget

to vote!

devmio